

AWS Cloud Security Attack Chain Reference

6 must-know attack scenarios for the Product Security Engineer interview

Interview Date: Friday, May 22, 2025

Covers: IAM PassRole · S3 Exfil · Lambda Secrets · CI/CD OIDC · Cross-Account Chaining · ECR Supply Chain

Attack chains index

- 01 IAM PassRole escalation**
Low-privilege user → full admin via iam:PassRole + EC2

- 02 Public S3 data exfiltration**
Misconfigured bucket → bulk PII theft → ransom

- 03 Lambda env var secret theft**
Overpermissive role → Lambda GetFunctionConfiguration → lateral movement

- 04 CI/CD pipeline OIDC token abuse**
Overpermissive OIDC trust → malicious PR → production backdoor

- 05 Cross-account role chaining**
Compromised dev account → prod account via unconstrained role assumption

- 06 Malicious container image (ECR supply chain)**
Poisoned base image → ECR → runtime credential theft from ECS task

Each chain is structured as: Kill chain steps → Blast radius → Detection signals (CloudTrail / GuardDuty) → Controls that break the chain → ProSec framing for interview answers.

CHAIN 01

IAM PassRole escalation

Low-privilege user → full admin via iam:PassRole + EC2

KILL CHAIN — STEP BY STEP

1 INITIAL ACCESS	Attacker gains low-privilege IAM credentials Leaked access key on GitHub, phishing, or misconfigured CI/CD secret. The key has limited permissions — not admin.
2 DISCOVERY	Enumerate IAM permissions Run <code>aws iam simulate-principal-policy</code> or <code>aws sts get-caller-identity</code> . Discovers the key has <code>iam:PassRole</code> and <code>ec2:RunInstances</code> — a classic escalation pair.
3 PRIVILEGE ESCALATION	Launch EC2 with high-privilege role attached Call <code>ec2:RunInstances</code> passing a high-privilege IAM role (e.g. <code>AdministratorAccess</code>) to the new instance via <code>iam:PassRole</code> . The attacker fully controls the new instance.
4 CREDENTIAL ACCESS	Steal role credentials via IMDS SSH into EC2 or exploit SSRF. Call <code>169.254.169.254/latest/meta-data/iam/security-credentials/</code> to retrieve the admin role's temporary STS credentials.
5 IMPACT	Full account takeover Use stolen credentials externally. Create new admin IAM users, disable CloudTrail, exfiltrate S3, pivot to other accounts via existing cross-account roles.
BLAST RADIUS	Full account compromise — attacker gains admin IAM role credentials usable from anywhere

DETECTION SIGNALS

Service	Detection signal
CloudTrail	<code>RunInstances</code> + <code>PassRole</code> from the same principal within minutes — this combination is a high-signal indicator
GuardDuty	<code>InstanceCredentialExfiltration.OutsideAWS</code> fires if credentials are used from a non-AWS IP address
CloudTrail	<code>CreateUser</code> or <code>AttachUserPolicy</code> called by an EC2 role — unusual principal type for IAM write operations

CONTROLS THAT BREAK THE CHAIN

Control	What it does	Breaks
Permission boundaries	Cap what roles can grant. Even with <code>PassRole</code> , the boundary prevents escalation above the boundary ceiling.	Breaks step 3

Control	What it does	Breaks
IMDSv2 + hop-limit=1	Enforce IMDSv2. hop-limit=1 blocks IMDS access if attacker reached instance via SSRF or proxy.	Breaks step 4
SCP: deny PassRole	SCP denying iam:PassRole for roles above a defined privilege level across the org.	Breaks step 3
EventBridge alert	Alert rule: RunInstances + PassRole by same principal within 5-min window triggers SNS page.	Detects step 3

PRODSEC ANGLE	iam:PassRole is the cloud equivalent of a setuid binary — a user delegates elevated authority to a process they control. Threat model any feature that creates EC2 or ECS resources: can the caller attach a role more powerful than their own?
---------------	---

CHAIN 02

Public S3 data exfiltration

Misconfigured bucket → bulk PII theft → ransom

KILL CHAIN — STEP BY STEP

1 RECONNAISSANCE	Attacker discovers public bucket via enumeration Tools like GrayhatWarfare, S3Scanner, or DNS enumeration. company-name-backup.s3.amazonaws.com lists objects with no authentication required.
2 COLLECTION	Bulk download all objects aws s3 sync s3://company-name-backup ./local/ — no credentials needed. Gigabytes of DB dumps, PII, config files downloaded within minutes.
3 CREDENTIAL ACCESS	Find secrets in downloaded files Downloaded files contain .env, config.yml, connection strings. Attacker extracts DB_PASSWORD, STRIPE_SECRET, JWT_SECRET, API keys — all in plaintext.
4 IMPACT	Ransom or sell data Attacker threatens public disclosure unless paid. Or auctions data on dark web. Extracted credentials used to pivot into RDS, payment systems, third-party APIs.
BLAST RADIUS	Customer PII / financial data exposed publicly — DPDP/GDPR breach, ransom demand, reputational damage

DETECTION SIGNALS

Service	Detection signal
CloudTrail	GetObject requests with no userIdentity (anonymous access) — S3 server access logs capture the source IP
Macie	PII discovery alert on bucket contents — Macie classifies data and alerts before the attacker finds it
S3 server logs	Sudden spike in GetObject requests from external IPs — sequential bulk access pattern

CONTROLS THAT BREAK THE CHAIN

Control	What it does	Breaks
S3 Block Public Access	Account-level + bucket-level public access block. Single highest-impact S3 control.	Breaks step 1
AWS Config rule	s3-bucket-public-read-prohibited auto-remediates any bucket that goes public.	Breaks step 1
SCP: deny PutBucketAcl	Org-wide SCP preventing any public ACL being set on any bucket.	Breaks step 1

Control	What it does	Breaks
Macie	Continuous PII scanning. Alerts on sensitive data classification before attacker downloads it.	Detects step 2

PRODSEC ANGLE	This is the cloud equivalent of an unauthenticated file share. In threat modelling: any feature that writes to S3 is a potential data exposure — ask 'what is the most sensitive thing this bucket could ever contain?' and design the controls for that worst case.
------------------	--

CHAIN 03

Lambda env var secret theft

Overpermissive role → Lambda GetFunctionConfiguration → lateral movement

KILL CHAIN — STEP BY STEP

1 INITIAL ACCESS	Attacker compromises a low-privilege role Via leaked key, SSRF on EC2/ECS, or compromised CI/CD token. Role has <code>lambda:ListFunctions</code> or <code>lambda:GetFunctionConfiguration</code> .
2 DISCOVERY	List all Lambda functions in the account <code>aws lambda list-functions</code> returns all function names, runtimes, and — if the role has <code>GetFunctionConfiguration</code> — environment variables in plaintext.
3 CREDENTIAL ACCESS	Dump all environment variables <code>aws lambda get-function-configuration --function-name fn-name</code> . <code>Environment.Variables</code> returns <code>DB_PASSWORD</code> , <code>API_KEY</code> , <code>STRIPE_SECRET</code> , <code>JWT_SECRET</code> in cleartext JSON.
4 LATERAL MOVEMENT	Use each extracted credential as a new attack surface <code>RDS</code> password connects directly to production database. <code>Stripe</code> key issues refunds or reads card data. <code>JWT secret</code> forges admin session tokens for the web application.
BLAST RADIUS	All secrets stored in Lambda env vars exposed — RDS, third-party APIs, payment gateways, JWT signing keys

DETECTION SIGNALS

Service	Detection signal
CloudTrail	<code>GetFunctionConfiguration</code> called by an unusual principal or from an unexpected source IP — especially if called at scale
CloudTrail	Bulk <code>GetFunctionConfiguration</code> across many functions in a short window — automated credential harvesting pattern
GuardDuty	Anomalous API call volume spike from a specific IAM role — baseline deviation alert

CONTROLS THAT BREAK THE CHAIN

Control	What it does	Breaks
AWS Secrets Manager	Never put secrets in env vars. Fetch from Secrets Manager at runtime — auto-rotation included.	Breaks step 3
Least-privilege IAM	Deny <code>lambda:GetFunctionConfiguration</code> except for ops roles that genuinely need it.	Breaks step 2

Control	What it does	Breaks
KMS-encrypt env vars	Encrypt Lambda env vars with a CMK. GetFunctionConfiguration returns ciphertext only — useless without KMS decrypt.	Breaks step 3
Custom Config rule	Flag Lambda functions with unencrypted environment variables containing secret-like key names.	Detects step 3

PRODSEC ANGLE

Treating Lambda env vars as a secrets store is the same anti-pattern as hardcoding credentials in source — but it's invisible until the IAM role is compromised. The threat model question: 'who can call GetFunctionConfiguration on this function?' If the answer is broad, every secret in that function is exposed.

CHAIN 04

CI/CD pipeline OIDC token abuse

Overpermissive OIDC trust → malicious PR → production backdoor

KILL CHAIN — STEP BY STEP

1 INITIAL ACCESS	Fork repo or submit malicious PR Attacker submits a PR to a repo whose GitHub Actions workflow has AWS OIDC integration. Or compromises a contributor account. The workflow runs on PRs.
2 PRIVILEGE ESCALATION	CI assumes overpermissive IAM role Workflow calls sts:AssumeRoleWithWebIdentity. Trust policy has no branch/repo condition — any PR from a fork can assume the role. Role has admin or deploy permissions.
3 PERSISTENCE	Inject backdoor into build artifact Malicious workflow step modifies application code before deployment. Inserts data exfiltration, reverse shell, or a hidden API endpoint. Artifact is signed and pushed.
4 IMPACT	Backdoor reaches production Every production request now executes attacker-controlled code. Attacker exfiltrates user data, harvests credentials, or uses the service as a C2 relay.
BLAST RADIUS	Attacker deploys arbitrary code to production — supply chain compromise of every user of the service

DETECTION SIGNALS

Service	Detection signal
CloudTrail	AssumeRoleWithWebIdentity from an unexpected GitHub repo or branch — check the FederatedUser token claim in the CloudTrail event
CloudTrail	UpdateFunctionCode or RegisterTaskDefinition from CI role on a non-main branch — flag immediately
CodeBuild	Build log anomalies — unexpected outbound connections, new packages not in requirements.txt, large binary blobs

CONTROLS THAT BREAK THE CHAIN

Control	What it does	Breaks
OIDC branch condition	Add Condition StringLike on token:sub or token:ref to restrict OIDC assumption to main branch only.	Breaks step 2
Least-privilege CI role	CI role has deploy permissions only — cannot read data, assume other roles, or create IAM entities.	Limits step 2
AWS Signer	Sign deployment artifacts. Lambda / ECS verify signature before execution — unsigned artifacts rejected.	Breaks step 3

Control	What it does	Breaks
Manual prod approval	Production deployment requires human approval gate in pipeline — cannot be automated from a PR.	Breaks step 3

PRODSEC ANGLE	This is a supply chain attack wearing a CI/CD uniform. Your prodsec threat model question: 'What is the blast radius if a PR author's GitHub account is compromised?' If that blast radius reaches production deployment, the OIDC trust policy needs a branch condition and the CI role needs privilege separation.
---------------	--

CHAIN 05

Cross-account role chaining

Compromised dev account → prod account via unconstrained role assumption

KILL CHAIN — STEP BY STEP

1 INITIAL ACCESS	Attacker compromises dev account credentials Dev account has weaker controls — no MFA enforcement, long-lived access keys, fewer SCPs. Attacker gains IAM credentials via phishing or leaked key.
2 DISCOVERY	Enumerate cross-account roles List IAM roles + attempt sts:AssumeRole. Finds a cross-account role in prod account with trust policy allowing dev account principals. No External ID required.
3 LATERAL MOVEMENT	Assume prod account role sts:AssumeRole into prod account. Attacker now holds temporary credentials in production, having crossed the account boundary without touching the internet.
4 COLLECTION	Access production data and resources Read prod RDS snapshots, S3 buckets, Secrets Manager secrets. Enumerate ECS tasks, EC2 instances, Lambda functions. Full prod data access.
5 PERSISTENCE	Create backdoor in prod account Create a new IAM user or role in prod with access keys. Even if dev credentials are rotated, attacker retains prod access via the backdoor entity.
BLAST RADIUS	Dev account breach becomes prod account breach — full lateral movement across the account boundary

DETECTION SIGNALS

Service	Detection signal
CloudTrail	AssumeRole in prod account with sourceAccount = dev account — cross-account calls appear in the TARGET account's CloudTrail
IAM Access Analyzer	Cross-account role without External ID flagged as externally accessible finding — review immediately
GuardDuty	Anomalous AssumeRole pattern — dev-account principal assuming prod roles at unusual hours or from unusual IPs

CONTROLS THAT BREAK THE CHAIN

Control	What it does	Breaks
External ID required	Require External ID in cross-account trust policy. Prevents confused deputy — attacker cannot supply the unknown External ID.	Breaks step 3

Control	What it does	Breaks
SCP on dev account	SCP denying sts:AssumeRole to prod account ARNs from within the dev account.	Breaks step 3
IAM Access Analyzer	Continuously finds roles with overly permissive cross-account trust policies. Review all findings weekly.	Detects step 2
Separate SCP tiers	Prod OU SCPs deny CreateUser, CreateAccessKey, and disabling logging — limits persistence options.	Limits step 5

PRODSEC ANGLE	Account boundaries are not trust boundaries unless you enforce them. The External ID requirement is the technical safeguard, but the architectural safeguard is treating every cross-account trust policy as a security-critical document requiring review. A misconfigured trust policy is effectively a privilege escalation path.
---------------	--

CHAIN 06

Malicious container image (ECR supply chain)

Poisoned base image → ECR → runtime credential theft from ECS task

KILL CHAIN — STEP BY STEP

1 SUPPLY CHAIN	Poison a base image or dependency Attacker compromises a public Docker Hub base image (e.g. node:18-alpine) or an npm/pip package used in the Dockerfile. A malicious layer exfiltrates env vars and calls credential endpoints on startup.
2 EXECUTION	Malicious image enters ECR via CI CI/CD pipeline pulls the compromised base image, builds on top of it, pushes to ECR. No image scanning or signing in place — ECR stores the poisoned image.
3 EXECUTION	ECS deploys poisoned task definition ECS pulls the image from ECR, starts the task. The malicious layer runs before application code — calls the ECS credential endpoint at 169.254.170.2 and exfiltrates to attacker C2.
4 CREDENTIAL ACCESS	Steal ECS task role credentials Malicious code calls <code>http://169.254.170.2/v2/credentials/{task-uuid}</code> — the ECS credential provider endpoint. Returns the task's IAM role credentials in plaintext JSON.
5 IMPACT	Lateral movement using task role If the ECS task role has broad permissions (S3, RDS, Secrets Manager, iam:PassRole), the attacker now has those permissions. Worst case: role chaining continues.

BLAST RADIUS	Attacker code runs in all production containers — ECS task role credentials stolen, full lateral movement from task permissions
--------------	--

DETECTION SIGNALS

Service	Detection signal
ECR / Inspector	Enhanced scanning on image push — known malicious package hashes or CVEs in base layer detected before deployment
GuardDuty	ECS Runtime Monitoring — unexpected outbound network connection from container to external IP at task startup
CloudTrail	GetObject or AssumeRole from ECS task role at unusual hours or accessing resources the task never normally touches

CONTROLS THAT BREAK THE CHAIN

Control	What it does	Breaks
ECR enhanced scanning	Enable Inspector-based scanning on every push. Define a deployment gate — block if critical CVEs found.	Breaks step 2

Control	What it does	Breaks
AWS Signer (image signing)	Sign images. Task definitions configured to reject unsigned images — poisoned images never run.	Breaks step 3
Task role least privilege	ECS task role scoped to minimum required. If no AWS API calls needed, no role attached — 169.254.170.2 returns nothing useful.	Limits step 4
GuardDuty ECS runtime	Detects unexpected syscalls, outbound connections, credential access from running containers in real time.	Detects step 3

PRODSEC
ANGLE

This is a classic supply chain attack — your prodsec instinct applies directly. 'What is the most dangerous thing this container is allowed to do?' Threat model the task role as carefully as you'd threat model an admin user. A task that only serves HTTP should not have S3 write or Secrets Manager read permissions.

Quick reference — chains at a glance

Chain	Root cause	Earliest break	Key detection
IAM PassRole escalation	iam:PassRole + ec2:RunInstances	Permission boundary	RunInstances + PassRole same principal
Public S3 exfiltration	No S3 Block Public Access	S3 Block Public Access	Anonymous GetObject in CloudTrail
Lambda env var theft	Secrets in env vars + broad IAM	Secrets Manager	Bulk GetFunctionConfiguration
CI/CD OIDC abuse	No branch condition on OIDC role	Branch condition + Signer	AssumeRoleWithWebIdentity from PR
Cross-account role chaining	No External ID on cross-acct role	External ID requirement	AssumeRole from dev → prod account
ECR supply chain / malicious img	No image scanning or signing	ECR enhanced scanning	GuardDuty ECS runtime outbound connection

Interview answer framework

Root cause	One sentence on the misconfiguration or over-permission that enabled the chain.
Attack steps	Walk through 3-4 steps in plain English — no jargon. Show you understand what the attacker actually does.
What the logs show	Name the specific CloudTrail event or GuardDuty finding. Interviewers love this precision.
Earliest break	Identify the single control that would have stopped the chain at step 1 or 2. Shows prevention > detection thinking.
Blast radius	State the worst-case impact clearly. Shows you think like a defender who understands risk, not just a technician.

Good luck on Friday. You've got this.